GAI is a flexible Go framework for building agent-style applications on top of LLMs.
It provides a generic interface for providers and models, prompt and context implementations, and a loop for agentic-calling workflows.

✨ Overview

The library is organized around three ideas:

- 🧩 `ai` defines the core provider, model, request, and response abstractions.
- 🗃️ `context` stores conversations, renders message history, and loads prompt files.
- 🔁 `loop` runs iterative model and tool execution when a model returns a tool call.

📋 Requirements

- Go `1.26.x` or newer
- API credentials for whichever provider you use

🚀 Quick Start

📦 Installation

```
go get github.com/lace-ai/gai
```

🧭 Usage

🛠️ Creating Providers and Models

PROF

To start, first create a provider. For example, for Gemini:

```
geminiProvider := gemini.New("your_api_key", nil)
```

▶ Details
🗃️ Use the Model Repository to manage multiple providers and dynamic model selection

You can use a `ModelRepository` to register multiple providers and look up models by name across providers.

```
modelRepo := ai.NewModelRepository(nil)
err := modelRepo.RegisterProvider(context.Background(), geminiProvider)
if err != nil {
```

```
    // handle error
}
```

To get a model from the repo just use the provider name and the model name:

```
model, err := modelRepo.GetModel(context.Background(), "gemini",
"gemini-3-flash-preview")
if err != nil {
    // handle error
}
```

💬 Generate Text

Now you can access models from that provider, and generate text:

```
model, err := geminiProvider.Model("gemini-3-flash-preview")
if err != nil {
    // handle error
}
response, err := model.Generate(context.Background(), ai.AIRequest{
    Prompt: ai.Prompt{
        System: "You are a helpful assistant.",
        Prompt: "What is the capital of France?",
    },
    MaxTokens: 100,
})
```

▶ Details

🔌 Implement Your Own Provider

PROF

Currently, the library includes Gemini and Mistral implementations. Gemini uses the official `go-genai` library, and Mistral uses direct HTTP calls to the Mistral API.

But you can implement your own provider by implementing the `Provider` and `Model` interfaces defined in the `ai` package.

Example:

Provider Implementation:

```
type MyProvider struct {
    // any configuration fields you need, e.g. API key
}

func (p *MyProvider) Name() string {
    return "myprovider"
```

```
}

func (p *MyProvider) Model(name string) (ai.Model, error) {
    // return a model implementation based on the name
}

func (p *MyProvider) ListModels() ([]string, error) {
    // return a list of available model names
}

func (p *MyProvider) Validate() error {
    // validate the provider configuration, e.g. check API key is set
}
```

Model Implementation:

```
type MyModel struct {
    // any configuration fields you need, e.g. model name, provider
reference
    name string
}

func (m *MyModel) Name() string {
    return m.name
}

func (m *MyModel) Generate(ctx context.Context, req ai.AIRequest)
(*ai.AIResponse, error) {
    // implement the logic to call your model API and return the
response
}

func (m *MyModel) GenerateStream(ctx context.Context, req ai.AIRequest)
<-chan ai.Token {
    // implement streaming token generation
}

func (m *MyModel) Close() error {
    // clean up any resources if needed
}
```

PROF

Now you can use your custom provider just like the built-in ones

🔁 Agentic Tool Calling

To build an agent with tools, use the `loop` package:

> [!TIP]
> Use an alias for the `context` package to avoid conflicts with `context` package from the standard

library. For example:

```
import aicontext "github.com/lace-ai/gai/context"
```

```
prompt := aicontext.NewPromptBuilder().
    System(aicontext.StaticPart(
        "base-system",
        "You are a helpful assistant that can call tools to get
information.",
    ).RequiredPart()).
    User(aicontext.StaticPart(
        "request",
        "What is the weather in New York?",
    ).RequiredPart())

l := loop.New(
    model, // the model you want to use
    []loop.Tool{myTool}, // any tools you want to provide, one echo tool
is included for testing
    prompt, // structured prompt builder from the context package
    nil, // optional tool response preprocessor, if nil the loop will
append tool results as-is
)

tokenCh, _, errCh := l.Loop(context.Background())
for range tokenCh {
    // handle streamed tokens
}
for err := range errCh {
    if err != nil {
        // handle error
    }
}
messages := l.Messages() // get final conversation messages, including
tool calls and responses

var builder strings.Builder
aicontext.RenderMessages(messages, &builder)
fmt.Println(builder.String()) // render the messages for display
```

PROF

▶ Details

🧩 Implement Your Own Tool

To implement your own tool, create a struct that implements the `Tool` interface:

```
type myToolArgs struct {
    Query string `json:"query"`
}
```

```
type MyTool struct {
    // any configuration fields you need
}

func (t *MyTool) Name() string {
    return "my_tool"
}

func (t *MyTool) Description() string {
    return "A tool that does something useful."
}

func (t *MyTool) Params() string {
    return `{"type":"object","required":["query"],"properties":{"query":
{"type":"string","description":"Search query"}}}`
}

func (t *MyTool) Function(req *ai.ToolCall) *loop.ToolResponse {
    var args myToolArgs
    if err := loop.DecodeToolArgs(req, &args); err != nil {
        return &loop.ToolResponse{Err: err}
    }

    // implement your tool logic here using args.Query
    return &loop.ToolResponse{Text: "result for: " + args.Query}
}
```

Then include an instance of your tool in the `loop.New(...)` call.

🧠 Session Management

To manage conversation history and build prompts from it, use the `context` package:

PROF

```
store := mySessionStore // your implementation of SessionStore (e.g. in-
memory, database, etc.)
sessionID := 1
prompt := aicontext.NewPromptBuilder().
    System(aicontext.StaticPart(
        "base-system",
        "You are a helpful assistant that can call tools to get
information.",
    ).RequiredPart()).
    ContextSource("history", aicontext.History(store, sessionID, 5),
true).
    User(aicontext.StaticPart(
        "request",
        "What is the weather in New York?",
    ).RequiredPart())
```

```
l := loop.New(
    model, // the model you want to use
    []loop.Tool{myTool}, // any tools you want to provide
    prompt, // prompt builder owns system, history, and user prompt
parts
    nil, // optional tool response preprocessor
)
tokenCh, _, errCh := l.Loop(context.Background())
for range tokenCh {
    // handle streamed tokens
}
for err := range errCh {
    if err != nil {
        // handle error
    }
}
```

▶ Details

🗄 Implement Your Own Session Store

To implement your own session store, please visit the SessionStore interface and implement the required methods.

🧱 Package Layout

```
ai/          Core abstractions: Provider, Model, AIRequest, AIResponse,
ModelRepository. With implementations for Gemini and Mistral.
context/     Context management: Conversation/session types, prompt
loading, message rendering
loop/        Agent loop, tool parsing, tool execution helpers
testutil/    Mocks used by tests
```

PROF

🧩 Core Concepts

🏢 Provider

A provider is responsible for exposing available models and validating its own configuration.
The shared interface is:

```
type Provider interface {
    Name() string
    Model(name string) (Model, error)
    ListModels() ([]string, error)
    Validate() error
}
```

Use `ModelRepository` when you want to register multiple providers and look up models by name.

🧠 Model

A model generates text from an `AIRequest` and can return either a complete `AIResponse` or a stream of tokens.

```
type Model interface {
    Name() string
    Generate(ctx context.Context, req AIRequest) (*AIResponse, error)
    GenerateStream(ctx context.Context, req AIRequest) <-chan Token
    Close() error
}
```

📝 Prompt and Request

`ai.Prompt` combines three flat strings that providers send upstream:

- `System`: system instructions
- `Context`: prior conversation or external context
- `Prompt`: the current (user) request

`Prompt.CombinedPrompt()` concatenates those parts onto one string in this order: system, context, prompt.
For agent loops, prefer building this value through `context.NewPromptBuilder()` so system, context, and user prompt content can be composed from structured parts and dynamic sources.

`AIRequest` currently contains:

- `Prompt`
- `MaxTokens` is ignored by some providers, and might be removed in future versions.

PROF

`AIResponse` returns:

- `Text`
- `InputTokens`
- `OutputTokens`

🌐 Providers

♊ Gemini

Package: `ai/gemini`

Constructor:

```
gemini.New(apiKey string, debug gai.DebugSink) *gemini.Provider
```

Known model names:

- `gemini-3-flash-preview`
- `gemini-2.5-flash`
- `gemini-3.1-flash-lite-preview`
- `gemini-2.5-flash-lite`

🌀 Mistral

Package: `ai/mistral`

Constructor:

```
mistral.New(apiKey string, debug gai.DebugSink) *mistral.Provider
```

Known model names:

- `mistral-small-latest`
- `mistral-medium-latest`
- `mistral-large-latest`

⚙ Configuration Note

This library does not read environment variables automatically.
Create the provider with the API key you want to use, then register it in the repository.

💾 Context and Sessions

The `context` package is not the standard library `context` package.
Import it with an alias such as `aicontext` to avoid name collisions.

PROF

```
import aicontext "github.com/lace-ai/gai/context"
```

📨 Messages

Messages have one of four roles:

- `system`
- `user`
- `assistant`
- `tool`

Each message wraps a `Content` implementation such as text, tool calls, or tool results, (you can also implement your own).
`RenderMessages` formats history as tagged blocks for prompt sources such as `History`.

💬 Conversation

`Conversation` is a minimal interface passed to dynamic prompt sources so they can inspect the current loop messages:

```
type Conversation interface {
    Messages() []Message
}
```

🗃️ SessionStore

`SessionStore` is an interface, not a built-in database implementation.
You provide your own store that can:

- create sessions
- fetch sessions and messages
- add one or many messages

🧱 PromptBuilder

`PromptBuilder` composes the full `ai.Prompt` from structured parts:

```
prompt := aicontext.NewPromptBuilder().
    System(aicontext.StaticPart("base-system", "Follow the system
policy.").RequiredPart()).
    ContextSource("history", aicontext.History(store, sessionID, 5),
true).
    ContextSource("rag", ragSource, false).
    User(aicontext.StaticPart("request", "Summarize the project
status.").RequiredPart())
```

PROF

Parts are rendered in fixed section order (`system`, `context`, `user`) and append order inside each section. Each part has a `Tokens` field for future token accounting, but the builder does not count, trim, or enforce token budgets yet.

Dynamic sources implement:

```
type Source interface {
    BuildParts(ctx context.Context, conv Conversation) ([]Part, error)
}
```

Required source failures stop prompt building. Optional source failures are skipped.

The default renderer is XML-like and can be replaced with a custom renderer.

🧭 History Sources

`History(store, sessionID, limit)` returns a prompt source that loads stored messages, renders them as a `history` part, and appends current loop messages as a `current-loop` part.

📄 Prompt Files

`LoadPromptFromFile` reads `.md` and `.txt` files, trims whitespace, and returns the prompt text.

🔄 Loop and Tools

The `loop` package is for agent-style execution where the model can request tool calls.

🔁 Loop

`loop.New(...)` creates a loop with:

- a model
- optional tools
- a structured prompt builder
- an optional tool-response preprocessor

The prompt builder is called on every loop iteration, so dynamic sources can include current loop messages, session history, summaries, RAG results, or other runtime context.

The loop stops when the model returns a normal response or when the maximum iteration count is reached.

🧰 Tool Interface

Tools must implement:

```
type Tool interface {
    Name() string
    Description() string
    Params() string
    Function(req *ai.ToolCall) *ToolResponse
}
```

Tool calls are expected to arrive as JSON with this shape:

```
{
  "type": "function",
  "name": "tool_name",
  "arguments": {
    "some": "value"
  }
}
```

Tool call IDs are generated internally by the runtime and are not model-controlled.

🧪 Helper Functions

- `DetectToolCallsInStream` detects tool-call JSON objects in streamed text tokens.
- `CallTool` runs a tool by name.
- `DecodeToolArgs` unmarshals tool arguments into a typed struct.
- `RenderToolSignatures` formats tool metadata for prompting.

❗ Errors

Common exported errors include:

- `ai.ErrProviderNotFound`
- `ai.ErrProviderAlreadyExists`
- `ai.ErrNilModelRepository`
- `loop.ErrModelNotConfigured`
- `loop.ErrToolNotFound`
- `loop.ErrMaxIterations`
- `context.ErrPromptMissing`
- `context.ErrSessionNotFound`
- `gemini.ErrInvalidAPIKey`
- `mistral.ErrInvalidAPIKey`

Handle provider and tool errors at the call site, especially when a model or session store is user-configured.

To see all the errors, check the errors.go file in each package.

🧪 Development

Run all tests:

PROF

```
go test ./...
```

Run a package test suite:

```
go test ./ai/...
go test ./loop/...
go test ./context/...
```

📝 Notes

- The `context` package name intentionally mirrors the domain it manages, but it is easy to confuse with `context.Context` from the standard library. Use an alias in imports. The context package is

likely to be renamed before official `1.0` release.

- `SessionManager` uses a default history window of 5 messages. Use `History(store, id, limit)` for an explicit limit.

🤝 Contributing

Contributions are welcome! Please open an issue or submit a pull request.
If you add a new provider or tool, document the new constructor, model names, and any required environment variables.

📜 Copyright and License

This library is licensed under the GNU LESSER GENERAL PUBLIC LICENSE v2.1. See LICENSE for details.